Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Vignesh **1BM24E 249**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

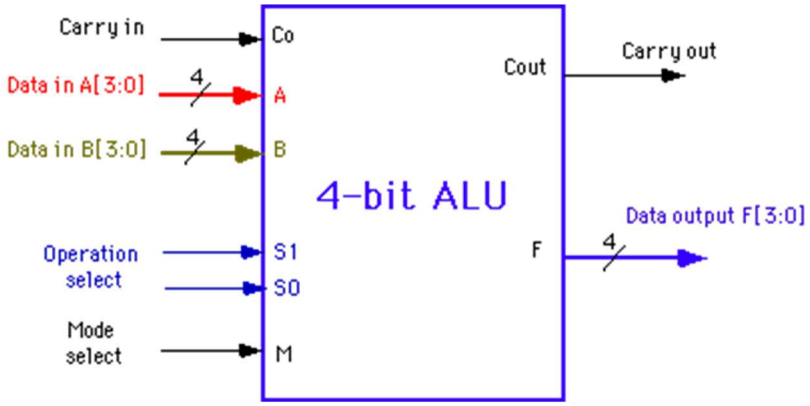
Academic Year

2026 – 2027

1. Arithmetic Logic Unit (ALU)

Design:

An 8-bit combinational ALU selected by a 3-bit opcode. It supports addition, subtraction, bitwise AND/OR/XOR, left shift, and right shift. The result is 8 bits; arithmetic carry/borrow behavior is not separately exposed.



Verilog HDL design code:

```
module alu (

input  wire [3:0] A,
input  wire [3:0] B,
input wire [2:0] OP,
output reg  [3:0] Y
);

 always @(*) begin
  case (OP)
    3'b000: Y = A + B;      // Addition
    3'b001: Y = A - B;      // Subtraction
    3'b010: Y = A & B;      // AND
    3'b011: Y = A | B;      // OR
    3'b100: Y = A ^ B;      // XOR
    3'b101: Y = ~A;         // NOT
    3'b110: Y = A << 1;     // Left shift
    3'b111: Y = A >> 1;     // Right shift
    default: Y = 4'b0000;
   endcase
 end
```

Testbench (TB):

```
`timescale 1ns/1ps

module alu_tb;

reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] Y;

alu uut (
   .A(A),
 . B(B),
 . sel(sel),
```

```
    . Y(Y)
);

initial begin
    A = 4'd5;
    B = 4'd3;

    sel = 3'b000; #10;
    $display("Addition = %d", Y);

    sel = 3'b001; #10;
    $display("Subtraction = %d", Y);

sel = 3'b010; #10;
    $display("AND = %b", Y);

    sel = 3'b011; #10;
    $display("OR = %b", Y);

    sel = 3'b100; #10;
    $display("XOR = %b", Y);

    sel = 3'b101; #10;
    $display("NOT = %b", Y);

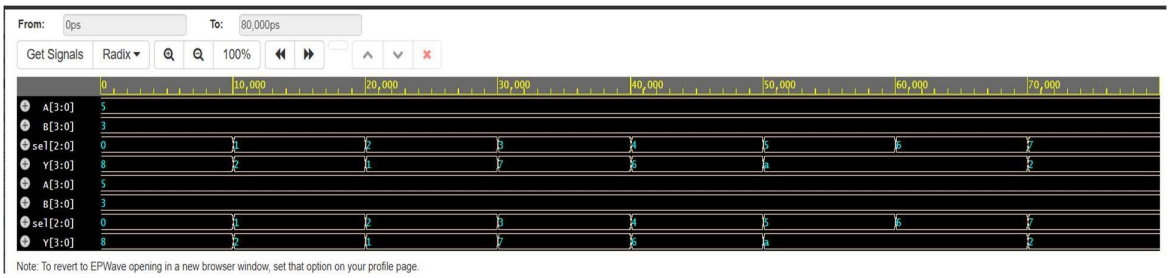
    sel = 3'b110; #10;
    $display("Left shift = %b", Y);

    sel = 3'b111; #10;
    $display("Right shift = %b", Y);

    $finish;
end
```

expected output

```
: time precision. This may cause confusing timing results.
: Affected design elements are:
:   -- module alu declared here: design.sv:2
Addition =  8
Subtraction =  2
AND = 0001
OR = 0111
XOR = 0110
NOT = 1010
Left shift = 1010
Right shift = 0010
testbench.sv:47: $finish called at 80000 (1ps)
```

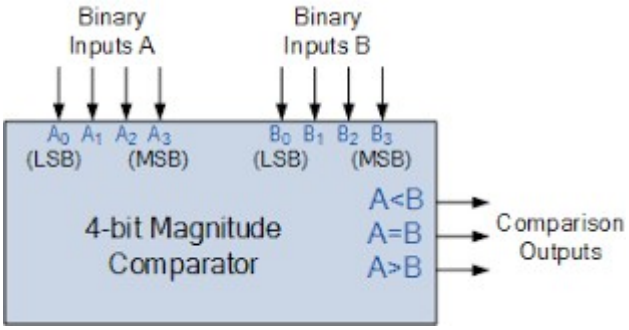


2.Comparator

Design

Compares two unsigned 4-bit inputs and asserts exactly one of gt, eq, or lt for each possible pair. A digital comparator is a combinational logic circuit that compares two binary numbers and determines whether the first number is greater than, less than, or equal to the second number.

This example implements a 4-bit magnitude comparator using Verilog HDL, with a design diagram, synthesizable code, testbench, and expected simulation output.



Verilog HDL design code

```
Modulecomparato
r(

   input [3:0] A, B,

    output reg GT,
LT, EQ

);

always @(*)
begin

    GT = 0;

    LT = 0;

    EQ = 0;


    if (A > B)

        GT = 1;

    else if (A < B)

        LT = 1;

    else

        EQ = 1;

end

endmodule
```

Testbench (TB)

```
module
comparator_tb;
```

```
reg [3:0] A, B;

wire GT, LT, EQ;

comparator uut(A,
B, GT, LT, EQ);

initial begin

   A = 5; B = 3;
#10;
display("A=%d
B=%d GT=%b
LT=%b EQ=%b",

            A, B, GT,
LT, EQ);

   A = 2; B = 6;
#10;

$display("A=%d
B=%d GT=%b
LT=%b EQ=%b",

            A, B, GT,
LT, EQ);
   A = 4; B = 4;
#10;
$display("A=%d
B=%d GT=%b
LT=%b EQ=%b",

            A, B, GT,
LT, EQ);

   $finish;

end

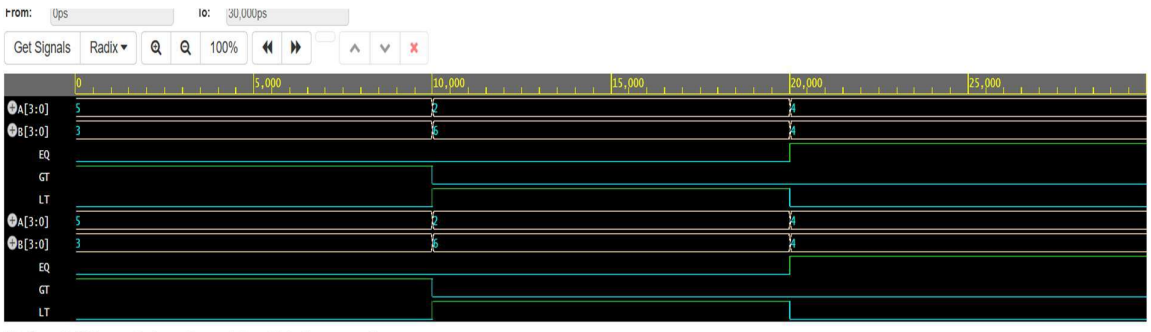
endmodule
```

Expected output

A= 5 B= 3 GT=1
LT=0 EQ=0

A= 2 B= 6 GT=0
LT=1 EQ=0

A= 4 B= 4 GT=0
LT=0 EQ=1

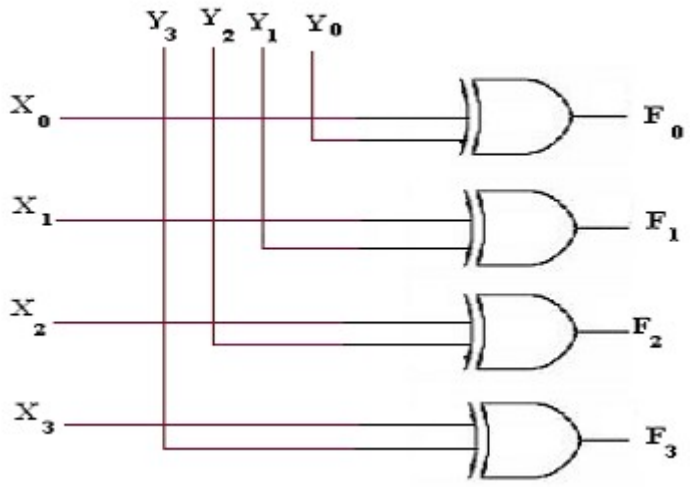


Note: To revert to FPWave opening in a new browser window, set that option on your profile page.

2. 4-Bit Parity Generator and Checker

Design

Generates even or odd parity for a 4-bit data word and checks a received 4-bit word plus its parity bit. For even parity, XOR of all data bits and parity bit should be 0.



Verilog HDL design code

```
  module parity(
 input [3:0] data,
 input mode,

 output parity_bit,
 output error
);

assign parity_bit = ^data ^ mode;
assign error = (^data ^ parity_bit) != mode;

endmodule
```

Testbench (TB)

```
 module parity_tb;
reg [3:0] data;
reg mode;
wire parity_bit, error;

parity uut(data, mode, parity_bit, error);

initial begin
 // Even parity
 data = 4'b1011;
 mode = 0;
 #10;
 $display("Data=%b Mode=%b Parity=%b Error=%b",
        data, mode, parity_bit, error);

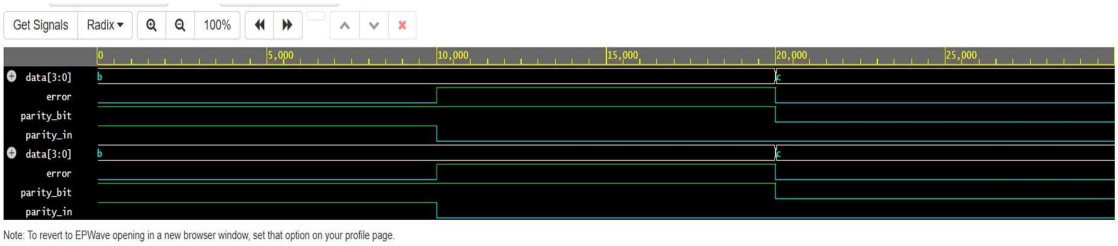
 // Odd parity
 data = 4'b1011;
 mode = 1;
 #10;
 $display("Data=%b Mode=%b Parity=%b Error=%b",
        data, mode, parity_bit, error);

 $finish;
end
```

endmodule

Expected output

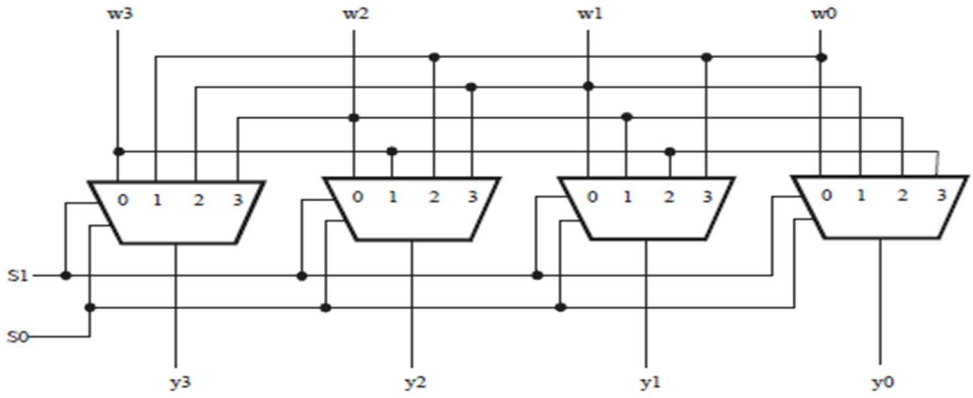
Data=1011 Mode=0 Parity=1 Error=0
Data=1011 Mode=1 Parity=0 Error=0



3. 4-Bit Barrel Shifter

Design

Combinational 4-bit logical barrel shifter. Direction 0 shifts left; direction 1 shifts right. The shift amount is 0–3 and zeros are shifted in.



Verilog HDL design code

```
 module barrel(
  input [3:0] data,
  input [1:0] shamt,
  input dir,
  output reg [3:0] y
);

always @(*) begin
  if (dir == 0)
    y = data << shamt;
  else
    y = data >> shamt;
end
endmodule
```

Testbench (TB)

```
  module barrel_tb;
```

```
reg [3:0] data;
reg [1:0] shamt;
reg dir;
wire [3:0] y;

barrel uut(data, shamt, dir, y);

initial begin
    // Left shift
    data = 4'b1011;
    shamt = 1;
    dir = 0;
    #10;
    $display("Left shift: data=%b shift=%d y=%b",
             data, shamt, y);

    // Left shift by 2
    shamt = 2;
    #10;
    $display("Left shift: data=%b shift=%d y=%b",
             data, shamt, y);

    // Right shift by 1
    shamt = 1;
    dir = 1;
    #10;
    $display("Right shift: data=%b shift=%d y=%b",
             data, shamt, y);

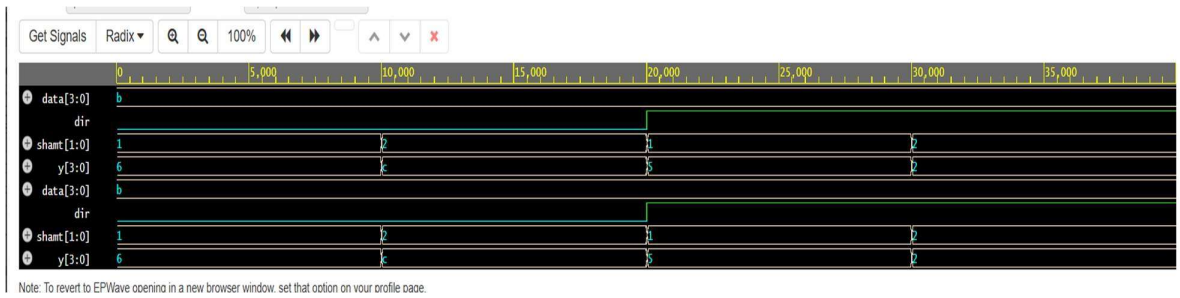
    // Right shift by 2
    shamt = 2;
    #10;
    $display("Right shift: data=%b shift=%d y=%b",
             data, shamt, y);

    $finish;
end

endmodule
```

Expected output

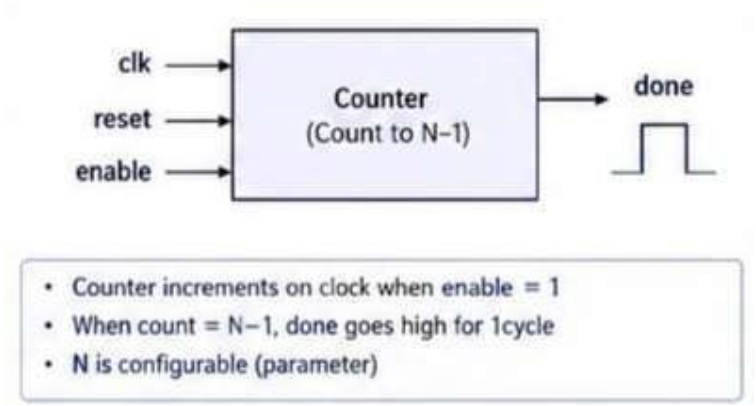
Left shift: data=1011 shift=1 y=0110
Left shift: data=1011 shift=2 y=1100
Right shift: data=1011 shift=1 y=0101
Right shift: data=1011 shift=2 y=0010



4. Timer Module

Design

Counts clock cycles and raises done for one clock cycle after COUNT_MAX cycles. Active-high asynchronous reset clears the counter and output.



- Counter increments on clock when enable = 1
- When count = N−1, done goes high for 1cycle
- N is configurable (parameter)

Verilog HDL design code

```
 module timer(
  input clk,
  input reset,
  output reg done,
  output reg [3:0] count
);

always @(posedge clk or posedge reset) begin
  if (reset) begin
     count <= 0;
     done <= 0;
  end
  else begin
     if (count == 9) begin
        count <= 0;
        done <= 1;
     end
     else begin
        count <= count + 1;
        done  <= 0;
     end

 end
end

endmodule
```

Testbench (TB)

```
 `timescale 1ns/1ps

module timer_tb;

reg clk, reset;
wire done;
wire [3:0] count;
timer uut(
  .clk(clk),
  .reset(reset),
  .done(done),
  .count(count)
);

// Generate clock: period = 10 ns
always #5 clk = ~clk;

initial begin
```

```
    clk = 0;
    reset = 1;
   #10;
    reset = 0;
   #120;
    $finish;
end

initial begin
    $monitor("Time=%0t  Count=%d  Done=%b",
            $time, count, done);
end
endmodule
```

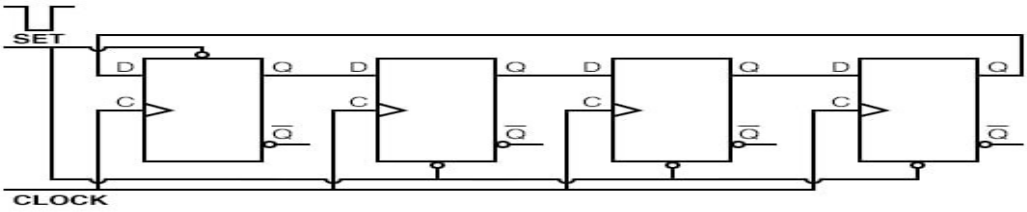
Expected output:

```
  Time=0     Count= 0 Done=0
Time=15000 Count= 1 Done=0
Time=25000 Count= 2 Done=0
Time=35000 Count= 3 Done=0
...
Time=95000 Count= 9 Done=0
Time=105000 Count= 0 Done=1
Time=115000 Count= 1 Done=0
```

5. Ring Counter

Design

Four-bit one-hot ring counter. A single 1 circulates through the register. Reset initializes the state to 0001.



Set one stage, clear three stages

Verilog HDL design code

```
module ring_counter(

  input clk,

   input reset,

   output reg [3:0] q

);

always @(posedge clk or posedge reset) begin

   if (reset)

      q <= 4'b0001;

   else

      q <= {q[2:0], q[3]};

end
```

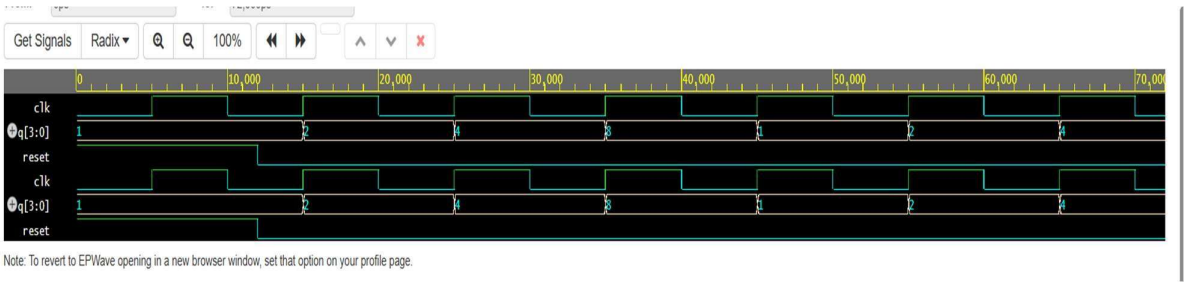
```
endmodule
```

Testbench (TB)

```
  `timescale 1ns/1ps
module ring_counter_tb;
reg clk, reset;
wire [3:0] q;
ring_counter uut(
    .clk(clk),
    .reset(reset),
    .q(q)
);
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset = 1;
    #10 reset = 0;
   60;
    $finish;
end

initial begin
    $monitor("Time=%0t q=%b", $time, q);
end
endmodule
```

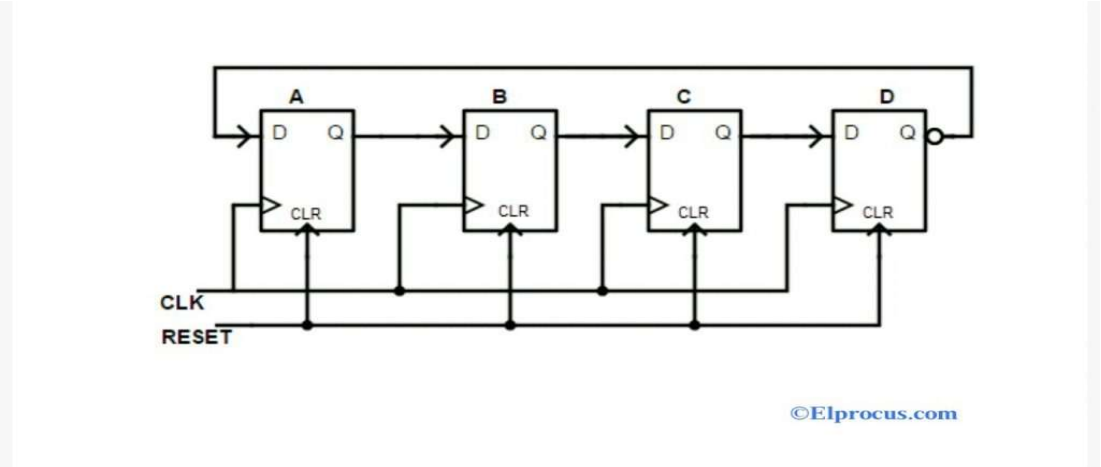
Expected output



6. Johnson Counter

Design

Four-bit Johnson counter (twisted ring counter). The inverted last bit is fed back into the first bit.



Verilog HDL design code

```
module johnson_counter(
    input clk,
    input reset,
    output reg [3:0] q
);
always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= {q[2:0], ~q[3]};
end
endmodule
```

Testbench (TB)

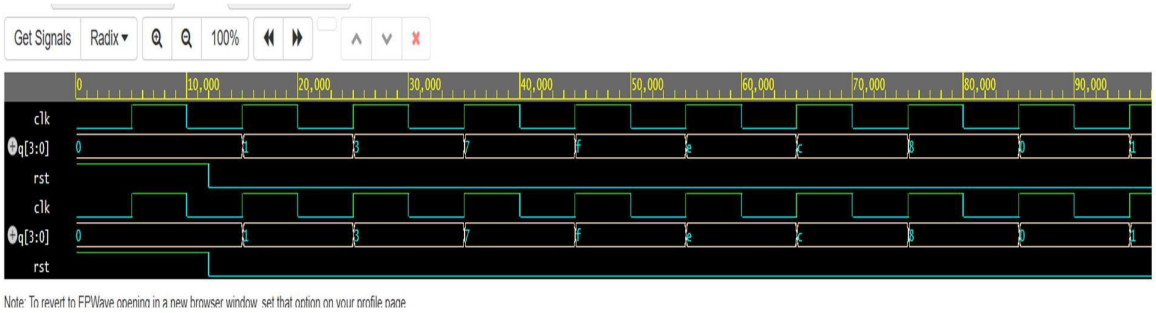
```
`timescale 1ns/1ps
module johnson_counter_tb;
reg clk, reset;
wire [3:0] q;
// Instantiate Johnson counter

johnson_counter uut(
    .clk(clk),
    .reset(reset),
    .q(q)
);
// Clock generation: 10 ns period
always #5 clk = ~clk;
```

```
initial begin
    // Waveform generation
    $dumpfile("johnson_counter.vcd");
    $dumpvars(0, johnson_counter_tb);
    clk = 0;
    reset = 1;
    #12 reset = 0;
    #80;
    $finish;
end

initial begin
    $monitor("Time=%0t reset=%b q=%b", $time, reset, q);
end
ndmodule
```

Expected output

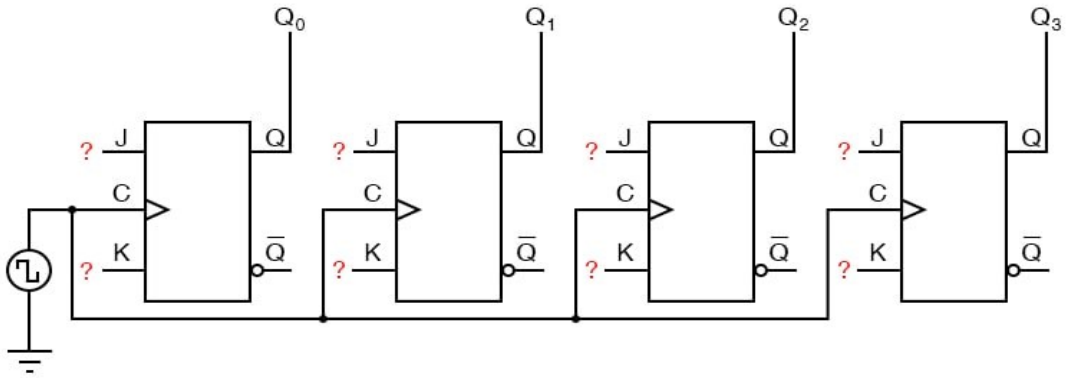


Note: To revert to FPWave opening in a new browser window, set that option on your profile page.

8A. Synchronous Counter

Design

Four-bit up-counter. All flip-flops are clocked by the same clock.



Verilog HDL design code

```
module sync_counter(
    input clk,
```

```
    input reset,
    output reg [3:0] q
);
always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= q + 1'b1;
end
endmodule
```

Testbench (TB)

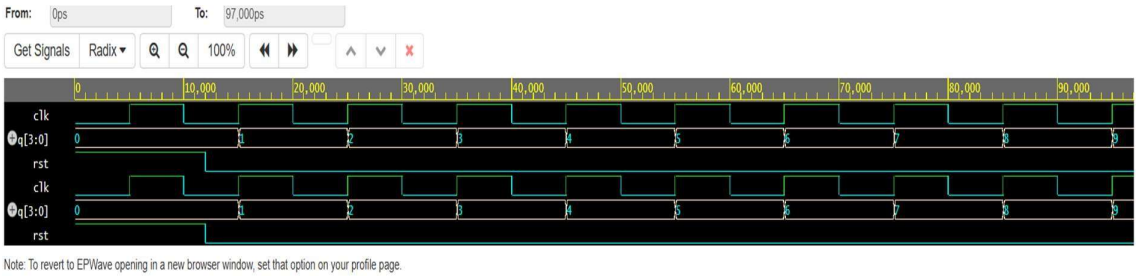
```
  `timescale 1ns/1ps
module sync_counter_tb;
reg clk, reset;
wire [3:0] q;
// Instantiate synchronous counter
sync_counter uut(
    .clk(clk),
    .reset(reset),
    .q(q)
);
// Clock generation: 10 ns period
always #5 clk = ~clk;
initial begin
    // Waveform generation
    $dumpfile("sync_counter.vcd");

   $dumpvars(0, sync_counter_tb);
    clk = 0;
    reset = 1;
    #12 reset = 0;
    #160;
    $finish;
end

initial begin
    $monitor("Time=%0t reset=%b q=%b", $time, reset, q);
end
```

endmodule

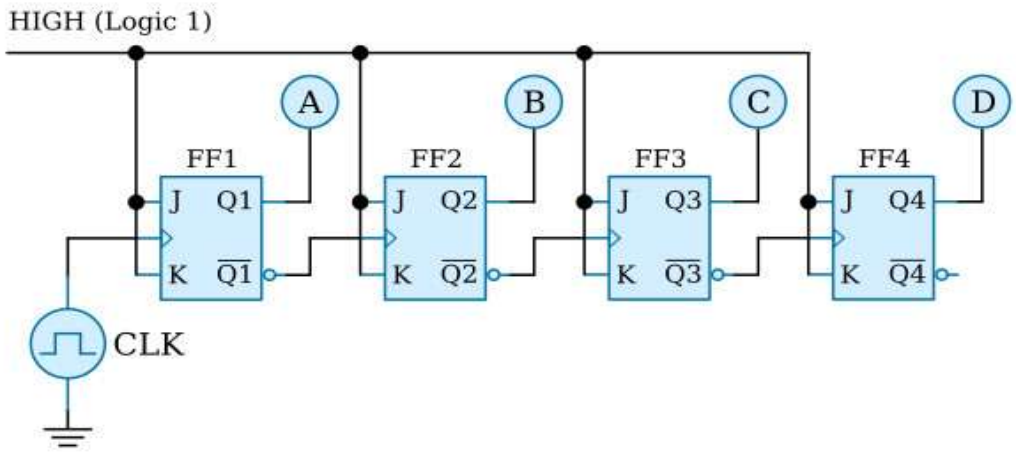
Expected output



B. Asynchronous (Ripple) Counter

Design

Four-bit ripple up-counter. The first bit is clocked by clk and each later bit is clocked by the preceding bit's falling edge. This demonstrates ripple timing; synchronous counters are generally preferred



Verilog HDL design code

```
 module ripple_counter(
    input clk,
    input reset,
    output reg [3:0] q
);
/ First flip-flop
always @(posedge clk or posedge reset) begin
    if (reset)
        q[0] <= 1'b0;

else
        q[0] <= ~q[0];
end
```

```
// Second flip-flop
always @(negedge q[0] or posedge reset) begin
    if (reset)
        q[1] <= 1'b0;
    else
        q[1] <= ~q[1];
end
// Third flip-flop
always @(negedge q[1] or posedge reset) begin
    if (reset)
        q[2] <= 1'b0;
    else
        q[2] <= ~q[2];
end
// Fourth flip-flop
always @(negedge q[2] or posedge reset) begin
    if (reset)
        q[3] <= 1'b0;
    else
        q[3] <= ~q[3];
end
endmodule
```

Testbench (TB)

```
 `timescale 1ns/1ps
module ripple_counter_tb;
reg clk, reset;
wire [3:0] q;

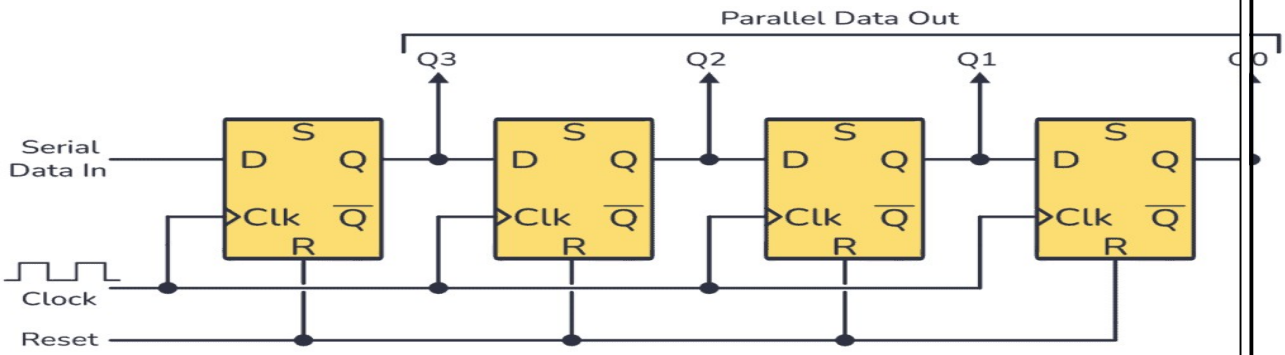
// Instantiate ripple counter
ripple_counter uut(
    .clk(clk),
    .reset(reset),
    .q(q)
);
// Clock generation: 10 ns period
always #5 clk = ~clk;
initial begin
    // Waveform generation
```

```
    $dumpfile("ripple_counter.vcd");
    $dumpvars(0, ripple_counter_tb);
    clk = 0;
    reset = 1;
    #12 reset = 0;
    #170;
    $finish;
end
initial begin
    $monitor("Time=%0t reset=%b q=%b", $time, reset, q);
end
endmodule
```

9. hift Register

Design

Four-bit serial-in, parallel-out (SIPO) shift register with synchronous shift enable and asynchronous reset.



Verilog HDL design code

```
module shift_register(
    nput clk,
    input reset,
    input serial_in,
    utput reg [3:0] q
);

always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= {serial_in, q[3:1]};
end
```

```
endmodule
```

Testbench (TB)

```
`timescale 1ns/1ps

module shift_register_tb;

reg clk, reset, serial_in;

wire [3:0] q;

// Instantiate shift register

shift_register uut(

    .clk(clk),

    .reset(reset),

    .serial_in(serial_in),

    .q(q)

);

// Clock generation: 10 ns period

always #5 clk = ~clk;


initial begin

    // Waveform generation

    $dumpfile("shift_register.vcd");

    $dumpvars(0, shift_register_tb);

    clk = 0;

    reset = 1;

    serial_in = 0;
    #12 reset = 0;
    // Apply serial input bits

    serial_in = 1; #10;

    serial_in = 0; #10;

    serial_in = 1; #10;

    serial_in = 1; #10;

    serial_in = 0; #10;

    #10;

    $finish;

end


initial begin

    $monitor("Time=%0t  reset=%b  serial_in=%b  q=%b",

            $time, reset, serial_in, q);

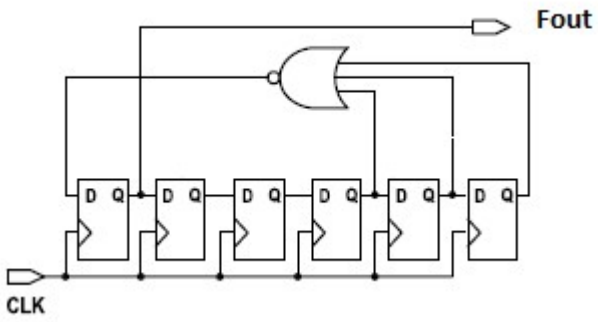
end
```

endmodule

10. Divide-by-N Counter

Design

Parameterized clock-enable pulse generator. tick is asserted for one clock cycle every N input clock cycles. It divides the event rate by N; it does not create a 50%-duty-cycle clock for arbitrary N.



Verilog HDL design code

```
module divide_by_n #(
    parameter N = 10
)(
    input clk,
    input reset,
    output reg clk_out
);
reg [31:0] count;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        count   <= 0;
        clk_out <= 0;
    end
    else begin
        if (count == N-1) begin
   count   <= 0;
            clk_out <= ~clk_out;
        end
        else begin
            count <= count + 1;
        end
    end
end
endmodule
```

Testbench (TB)

```
`timescale 1ns/1ps
module divide_by_n_tb;
reg clk, reset;
wire clk_out;

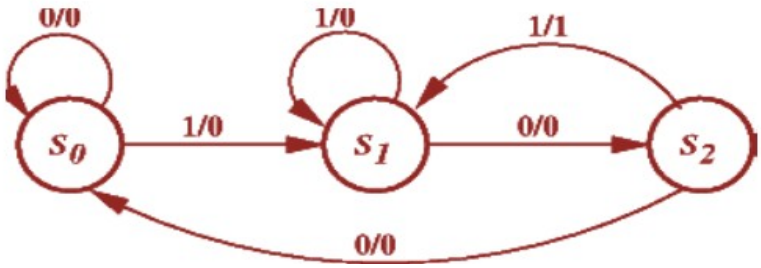
// Instantiate divide-by-10 counter
divide_by_n #(.N(5)) uut (
    .clk(clk),
    .reset(reset),
    .clk_out(clk_out)
);
// Input clock: 10 ns period
always #5 clk = ~clk;
initial begin
    // Waveform generation
    $dumpvars(0, divide_by_n_tb);
    clk = 0;
    reset = 1;
    #12 reset = 0;
    #200;
    $finish;
end

initial begin
    $monitor("Time=%0t  reset=%b  clk_out=%b",
            $time, reset, clk_out);
end
endmodule
```

11. Mealy FSM — Sequence Detector for 101(combinational module)

Design

Detects the bit pattern 101 in a serial input. The output can assert on the same clock edge as the final input bit because it depends on current state and input (Mealy output). Overlapping matches are supported.



Present States		i/p	Next States		Flip Flop Excitations		O/P
X	Y		X'	Y'	Dx	Dy	
0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0
0	1	0	0	0	0	0	0
0	1	1	0	0	0	0	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	X	X	X	X	X
1	1	1	X	X	X	X	X

Verilog HDL design code

```
module mealy_101(
  input clk,
  input reset,
  input din,
  output reg detect
);

reg [1:0] state;

parameter S0 = 2'b00,
          S1 = 2'b01,
          S2 = 2'b10;

always @(posedge clk or posedge reset) begin
  if (reset)
    state <= S0;
  else begin
    case (state)
      S0: begin
        if (din)
          state <= S1;
        else
          state <= S0;
      end

      S1: begin
        if (din)
          state <= S1;
        else
          state <= S2;
      end

      S2: begin
        if (din)
```

```
                state <= S1;
            else
                state <= S0;
        end
  default: state <= S0;
    endcase
  end
end

always @(*) begin
  detect = 0;

  case (state)
    S2: begin
      if (din)
        detect = 1;
    end

    default: detect = 0;
  endcase
end
endmodule
```

Testbench (TB)

```
`timescale 1ns/1ps

module mealy_101_tb;

reg clk, reset, din;
wire detect;
mealy_101 uut(
  .clk(clk),
  .reset(reset),
  .din(din),
  .detect(detect)
);

// Clock generation
always #5 clk = ~clk;

initial begin
  // Waveform generation
  $dumpfile("mealy_101.vcd");
  $dumpvars(0, mealy_101_tb);

  clk = 0;
  reset = 1;
  din = 0;
  #12 reset = 0;

  // Input sequence: 1 0 1
  din = 1; #10;
  din = 0; #10;
  din = 1; #10;
  // Continue with additional bits
  din = 1; #10;
  din = 0; #10;
  din = 1; #10;

  #10;
  $finish;
end
```

```
initial begin
   $monitor("Time=%0t  din=%b  detect=%b",

      $time, din, detect);
end

endmodule
```

EXPECTED OUTPUT
Input x : 1 0 1 0 1 1 0
Output z : 0 0 1 0 1 0 0

12. Moore FSM — Sequence Detector for 101(sequence module)

Design

Detects 101 using a Moore output state. The detected output is high while the FSM is in the DETECT state, so it changes after the clock edge that completes the pattern.

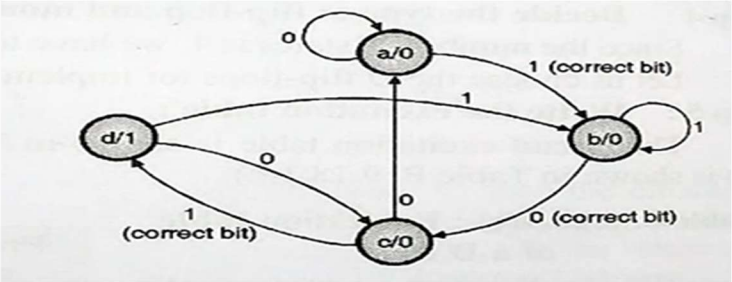


Fig3. State diagram for 101 detector

Verilog HDL design code

```
module moore_101 (
  input wire clk,
  input wire rst,
  input wire x,
  output wire z
);

  reg [1:0] state, next_state;

  localparam S0 = 2'b00,
             S1 = 2'b01,
             S2 = 2'b10,
             S3 = 2'b11;

  // Sequential state register
  always @(posedge clk or posedge rst) begin
     if (rst)
        state <= S0;
     else
        state <= next_state;
  end

  // Combinational next-state logic
  always @(*) begin
     next_state = S0;

     case (state)
        S0: begin
           if (x)
              next_state = S1;
           else
              next_state = S0;
        end
```

```
            S1: begin
                if (x)
                    next_state = S1;
                else
                    next_state = S2;
            end

            S2: begin
                if (x)
                    next_state = S3;lse

next_state = S0;   S3: begin if (x)
                    next_state = S1;
                else
                    next_state = S2;
             end

            default: next_state = S0;
          endcase
      end

      // Moore output depends only on current state
      assign z = (state == S3);

  endmodule
```

TESTBENCH(TB)

```
`timescale 1ns/1ps
module tb_moore_101;
    reg clk;
    reg rst;
    reg x;
    wire z;

    // Instantiate the DUT
    moore_101 uut (
        .clk(clk),
        .rst(rst),
        .x(x),
        .z(z)
    );

    // Clock: 10 ns period
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // VCD waveform dump
    initial begin
        $dumpfile("moore_101.vcd");
        $dumpvars(0, tb_moore_101);
    end

    // Monitor signals
    initial begin
        $timeformat(-9, 0, " ns", 6);
        $monitor("Time=%0t clk=%b rst=%b x=%b z=%b state=%b",
                 $time, clk, rst, x, z, uut.state);
    end
```

```
    // Test input sequence: 1010110
    initial begin
        rst = 1;
        x = 0;

        #12 rst = 0;

        @(negedge clk); x = 1;
        @(negedge clk); x = 0;
        @(negedge clk); x = 1;
        @(negedge clk); x = 0;
        @(negedge clk); x = 1;
        @(negedge clk); x = 1;
        @(negedge clk); x = 0;

        repeat (2) @(negedge clk);
        $finish;
    end

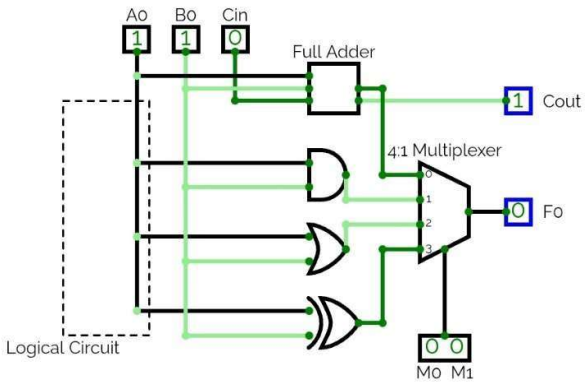
endmodule
```

EXPECTED OUTPUT
Input x : 1 0 1 0 1 1 0
Output z : 0 0 1 0 1 0 0

13. Random Sequence Generator — 4-Bit LFSR

Design

A 4-bit linear-feedback shift register generates a repeatable pseudo-random sequence. This example uses feedback q[3] XOR q[2] and a nonzero seed. It is deterministic and must not be used for cryptography.



Verilog HDL design code

```
module lfsr_4bit (
    input wire clk,
    input wire reset,
    output reg [3:0] random_out
);
    wire feedback;
    assign feedback = random_out[3] ^ random_out[0];
```

```
    always @(posedge clk) begin
        if (reset)
            random_out <= 4'b0001;  // Initial seed
        else
            random_out <= {random_out[2:0], feedback};
    end
endmodule
```

TESTBENCH

```
`timescale 1ns / 1ps
module tb_lfsr_4bit;
   reg clk;
    reg reset;
    wire [3:0] random_out;
    // Instantiate DUT
    lfsr_4bit uut (
        .clk(clk),
        .reset(reset),
        .random_out(random_out)
    );
    // Generate clock: 10 ns period
    always #5 clk = ~clk;
    // Test sequence
    initial begin
        // Waveform dump
        $dumpfile("lfsr_4bit.vcd");
        $dumpvars(0, tb_lfsr_4bit);
        // Initialize signals
        clk = 0;
        reset = 1;
        // Apply reset
        #10;
        reset = 0;
        // Run for 20 clock cycles
        repeat (20) begin
            @(posedge clk);
            #1;
            $display("Time = %0t | Random Output = %b",
```

```
                $time, random_out);
        end
        #10;
        $finish;
    end
endmodule
```